GRODEALZ: AN AI-POWERED MULTI-STORE GROCERY BASKET OPTIMIZER AND MERCHANT QUOTATION PLATFORM

A project report submitted in partial fulfilment for the course
Project - II (CS4202)

Submitted By

PRINCE KUMAR DUBEY
Roll No: 230101081
Semester - VII

Under the Supervision of

DR. KISHORJIT NONGMEIKAPAM
Associate Professor



Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

November, 2026

Declaration

I hereby declare that the work presented in this project report entitled **"GRODEALZ: AN AI-POWERED MULTI-STORE GROCERY BASKET OPTIMIZER AND MERCHANT QUOTATION PLATFORM"** is my own bonafide work carried out under the supervision of **Dr. Kishorjit Nongmeikapam**, Associate Professor, Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur.

I have strictly adhered to the principles of academic honesty and integrity. I have properly referenced and cited all ideas, methodologies, and source materials borrowed from external publications, open-source libraries, or online documents. No part of this report has been plagiarised or submitted anywhere else for the award of any other degree or diploma.

PRINCE KUMAR DUBEY
Roll No: 230101081
Department of Computer Science and Engineering
IIIT Senapati, Manipur
Date: November, 2026

Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Kishorjit Nongmeikapam Email: kishorjit@iiitmanipur.ac.in
Associate Professor Department of Computer Science and Engineering

Certificate

This is to certify that the project report entitled **"GRODEALZ: AN AI-POWERED MULTI-STORE GROCERY BASKET OPTIMIZER AND MERCHANT QUOTATION PLATFORM"** submitted to the Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fulfilment for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** is an authentic record of bonafide work carried out by **PRINCE KUMAR DUBEY** (Roll No: **230101081**).

The candidate has satisfactorily completed all requirements of Project - II (CS4202), and the results presented in this report have been verified and found satisfactory.

Signature of Supervisor

(Dr. Kishorjit Nongmeikapam)

Signature of Examiner 1: ..... Date: .....

Signature of Examiner 2: ..... Date: .....

Signature of Examiner 3: ..... Date: .....

Signature of Examiner 4: ..... Date: .....

Abstract

In today's quick-commerce market, buying groceries online comes with two major everyday frustrations. First, customers who want the best prices or have diverse shopping lists usually have to check multiple stores manually to find who has fresh stock and lower rates. Buying everything from one shop is convenient but often expensive, while splitting orders manually across two or three shops takes too much effort. Second, when people need bulk items or custom requirements—such as twenty-five kilograms of rice for a hostel with a specific budget and deadline—standard grocery apps do not allow them to simply post what they need. Instead, users are forced to search for fixed SKUs with fixed prices, while local merchants have no way to bid or offer competitive rates.

To solve these problems, this project develops and implements **GroDealz**, a complete web platform built on the MERN stack (MongoDB, Express.js, React 18, Node.js) combined with a Python-based autonomous agent service. GroDealz introduces two main innovations:

1. **Autonomous AI Grocery Shopping Agent (Module A):** Users describe their needs in plain English or Hinglish (e.g., *"Party snacks and beverages for 5 people under Rs. 600"*). The system extracts the intent, family size, dietary constraints, and budget, queries available local stores (such as FreshMart Central, QuickBazaar, Organic Harvest, and Paona Supermarket), checks real-time stock, and builds an optimal shopping basket. It compares single-store convenience against multi-store splitting, proving real savings (for instance, saving Rs. 267 on a typical weekly order even after paying separate delivery fees), and provides a visual store journey audit.

2. **Reverse RFQ Marketplace and Merchant Quotation Engine (Module B):** Customers post shopping requests in natural language. The system extracts items, quantities, target budgets, and delivery deadlines. Nearby store owners can see open requests, verify their stock, and submit transparent, itemized line-item quotations. The quotation engine dynamically ranks competing offers according to the buyer's choice: lowest total price, fastest delivery, highest store rating, or a balanced score. Buyers can also send counter-offers, and merchants can revise their quotes. Once accepted, the system converts the quote directly into a placed order.

In addition, GroDealz features a hybrid product recommendation engine (combining collaborative filtering, content features, and time-of-day scoring), an hourly demand forecasting cron job, real-time status updates via Socket.IO, and an AI-assisted customer support desk with automatic intent tagging and PII masking. The platform has been tested thoroughly with automated test suites and deployed on Render with a live database.

Acknowledgement

First and foremost, I would like to express my deepest gratitude to my project supervisor, **Dr. Kishorjit Nongmeikapam**, Associate Professor, Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur, for his constant encouragement, constructive criticism, and technical guidance throughout the progress of this project. His practical suggestions regarding system architecture, algorithm design, and real-world applicability helped me shape this work into a working software system.

I sincerely thank **Dr. Navanath Saharia**, Head of the Department of Computer Science and Engineering, for providing departmental facilities, infrastructure, and continuous support for student project work.

I am also thankful to all faculty members and staff of the Department of CSE at IIIT Manipur for their guidance and for creating an inspiring academic environment.

I want to thank my classmates and friends for testing the web application, pointing out edge cases in the user interface, and providing feedback during testing.

Finally, I express my heartfelt thanks to my parents and family for their unconditional love, moral support, and patience throughout my studies.

PRINCE KUMAR DUBEY
Roll No: 230101081
B.Tech CSE, Semester - VII
IIIT Senapati, Manipur

Contents

List of Figures **v**

List of Tables **vi**

List of Abbreviations **1**

1 Introduction **2**

1.1 Problem Statement 3
1.2 Objectives of the Project 3
1.3 Scope of the Project 4
1.4 Regional Market Context and Hyper-Local Economics 5
1.5 Project Development Methodology and Iterative Lifecycle 6
1.6 Structure of the Report 6

2 Literature Survey **8**

2.1 Evolution of Online Grocery and Quick-Commerce 8
2.2 Recommendation Systems in Retail 8
2.2.1 Collaborative Filtering 9
2.2.2 Content-Based Filtering 9
2.2.3 Hybrid Approaches 9
2.3 Demand Forecasting in Perishable Retail 9
2.4 Autonomous Agents and Multi-Step Task Execution 10
2.5 Reverse Auctions and Multi-Attribute Request for Quotation (RFQ) Systems 11
2.6 Data Security, Privacy, and Transactional Concurrency in Multi-Sided Platforms 12
2.7 Comparative Summary 12

3 Requirement Engineering **14**

3.1 Stakeholder Analysis 14
3.2 Functional Requirements 14
3.2.1 Authentication and User Profiles (FR-AUTH) 14
3.2.2 Product Catalog and Discovery (FR-CATALOG) 15

3.2.3 Autonomous AI Shopping Agent and Multi-Store Optimizer (FR-AGENT) . . . 15
3.2.4 Reverse RFQ Marketplace and Merchant Quotation Engine (FR-RFQ) . . . 15
3.2.5 Order Lifecycle and Real-Time Tracking (FR-ORDER) . . . 16
3.2.6 Demand Prediction and Store Operations (FR-DEMAND) . . . 16
3.2.7 AI Customer Support CRM (FR-SUPPORT) . . . 16
3.3 Non-Functional Requirements . . . 17
3.4 Core Use Case Scenarios . . . 17
3.4.1 Use Case 1: Autonomous Multi-Store Grocery Basket . . . 17
3.4.2 Use Case 2: Custom Requirement Reverse RFQ and Merchant Quotation . . . 17

4 System Design **19**
4.1 Architectural Evolution: 1-Tier, 2-Tier, and 3-Tier Models . . . 19
4.1.1 1-Tier vs. 2-Tier Architectural Models . . . 19
4.1.2 3-Tier Architecture of GroDealz . . . 21
4.2 Database Design and Data Models . . . 22
4.2.1 Buyer Request Schema (`BuyerRequest`) . . . 22
4.2.2 Merchant Quotation Schema (`MerchantQuotation`) . . . 23
4.3 Autonomous AI Shopping Agent Workflow . . . 24
4.4 Module B Quotation Engine Design . . . 25
4.4.1 Itemized Pricing Calculations . . . 25
4.4.2 Multi-Criteria Dynamic Ranking Formulations . . . 25
4.5 Real-Time WebSocket Event Architecture . . . 26

5 Implementation **27**
5.1 Technology Stack . . . 27
5.2 Core Backend Services . . . 28
5.2.1 Natural Language Requirement Parser (`rfqParser.js`) . . . 28
5.2.2 Merchant Quotation Engine (`quotationEngine.js`) . . . 28
5.2.3 Hybrid Recommendation Subsystem . . . 29
5.2.4 Automated Demand Prediction . . . 29
5.3 Frontend Architecture and Components . . . 29
5.3.1 Customer Shopping Requirements Page (`BuyerRFQ.js`) . . . 29
5.3.2 Merchant Quotation Dashboard (`MerchantRFQ.js`) . . . 30
5.3.3 Autonomous AI Grocery Planner (`AIGroceryPlanner.js`) . . . 30
5.3.4 Customer Support CRM (`Support.js`) . . . 30
5.3.5 Homepage and Product Catalog (`Home.js` and `Products.js`) . . . 30
5.4 Autonomous LangGraph Decision Graph Implementation . . . 30
5.4.1 State Schema Specification (`AgentState`) . . . 30

5.4.2 Graph Node Pipeline and Execution Flow 31
5.5 Transaction Management and Atomic Quotation Acceptance 32
5.6 Real-Time WebSocket Event Multiplexing 32
5.7 Security Architecture and Sensitive Data Protection 33

6 Results and Interface Walkthrough **34**
6.1 Interface Walkthrough and Screenshots 34
6.1.1 GroDealz Home and Personalized Recommendations 34
6.1.2 Autonomous AI Grocery Planner and Store Journey Audit 35
6.1.3 Custom Shopping Requirements and Reverse RFQ Marketplace . 36
6.1.4 AI Customer Support CRM and Ticket Triage 37
6.2 Automated Test Suite Verification 38
6.3 Quotation Ranking Sensitivity Analysis 39
6.4 System Performance and Latency 40

7 Conclusion and Future Scope **42**
7.1 Summary of Contributions 42
7.2 Practical Impact and Regional Relevance 43
7.3 Current Limitations 43
7.4 Future Scope 44

Bibliography **45**

List of Figures

4.1 Comparison of 1-Tier and 2-Tier Architectural Models with Functional Descriptions 20

4.2 3-Tier High-Level Architecture of the GroDealz Platform. Tier 1 manages customer interactions in React 18, Tier 2 processes quotation algorithms and AI agents in Node.js/Python, and Tier 3 secures relational and document data in MongoDB Atlas. 21

4.3 Entity-Relationship Diagram (ERD) of GroDealz Database. Illustrates relational dependencies across seven primary collections including User, Store, Product, Order, BuyerRequest, MerchantQuotation, and Customer Support. 22

4.4 UML Class Diagram of System Architecture. Details domain model schemas, controller interfaces, algorithmic quotation engines, and real-time Socket.IO notification rooms. 24

6.1 GroDealz Main Interface Featuring Personalized Recommendation Rails and Freshness Badges. Shows authenticated customer view, dynamic recommendation tags, categorized grocery items, and freshness confidence indicators. 35

6.2 Autonomous Shopping Agent Interface with Store Journey Audit and Multi-Store Savings Proof. Illustrates natural language budget parsing, automated cross-store stock picking, and a calculated Rs. 267 net saving over single-store shopping. 36

6.3 Customer Custom Shopping Requirements Interface with AI Line-Item Extractor and RFQ Controls. Demonstrates NLP parsing of bulk hostel rice requests, budget sharing toggles, ranking priority selectors, and instant merchant broadcast. 37

6.4 AI Customer Support Dashboard with Automated Intent Classification and PII Masking. Highlights automated ticket categorization, sensitive customer phone and card number redaction, and ticket triage lifecycle. . 38

List of Tables

2.1 Comparison of GroDealz with Existing Grocery Delivery Approaches . . 13

6.1 Automated Test Suite Execution Results for Module B 39
6.2 Dynamic Ranking Output Across Different Customer Priorities 40
6.3 Measured System Response Times and Event Latency 40

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
B2C	Business to Consumer
CF	Collaborative Filtering
COD	Cash on Delivery
CORS	Cross-Origin Resource Sharing
CRM	Customer Relationship Management
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
FR / NFR	Functional / Non-Functional Requirement
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
NLP	Natural Language Processing
ODM	Object Document Mapper (Mongoose)
PII	Personally Identifiable Information
REST	Representational State Transfer
RFQ	Request for Quotation (Reverse Marketplace)
SKU	Stock Keeping Unit
SPA	Single-Page Application
UI / UX	User Interface / User Experience
URI / URL	Uniform Resource Identifier / Locator

Chapter 1

Introduction

Everyday grocery shopping is an essential activity for every household. Over the last three to four years, quick-commerce platforms like Blinkit, Zepto, and Instamart have made home delivery very popular across Indian cities. Most of these platforms focus on delivering small orders in ten to twenty minutes from their own private warehouses, usually called dark stores.

While this rapid delivery model is very convenient for quick emergency purchases, it has clear drawbacks when applied to general household shopping or regional markets such as Imphal:

1. **Single-store lock-in:** When a customer opens a typical grocery app, all items must come from that single warehouse. If that store does not have fresh tomatoes or runs out of cooking oil, the customer has to place a completely separate order on another app or visit a physical shop.

2. **No multi-store comparison:** In reality, different local shops offer better prices on different items. For example, a local supermarket might sell staples like wheat flour and basmati rice at wholesale rates, while a fresh organic vendor nearby sells vegetables at half the warehouse price. Manually opening three different apps, checking stock, calculating delivery fees, and placing three separate orders is tedious and time-consuming.

3. **Inability to post custom or bulk requirements:** If a student living in a hostel or a family planning an event needs twenty-five kilograms of rice, five litres of mustard oil, and fifty kilograms of potatoes within a specific budget of Rs. 2,500, existing apps offer no way to simply post this requirement. Customers are forced to browse fixed packages at fixed retail prices, and nearby neighborhood merchants have no mechanism to compete or provide bulk quotations.

4. **Manual search fatigue:** Entering every grocery item into a search bar one by one takes ten to fifteen minutes every week. Most people already know what they want in plain terms (e.g., *"Groceries for a family of four for three days under Rs. 1,500"*), but apps force them through a manual search process.

To address these practical challenges, this project develops **GroDealz**, an AI-powered grocery shopping platform that combines traditional e-commerce with an autonomous multi-store shopping agent and a reverse merchant quotation engine.

1.1 Problem Statement

The primary problems addressed in this project can be divided into three specific issues:

1. **Manual Multi-Store Optimization Problem:** A customer wanting to buy a full basket of groceries at the lowest cost must manually compare prices and stock across multiple local vendors. A naive multi-store order might end up costing more if each shop charges a separate delivery fee. The system needs an autonomous optimizer that checks prices, verifies fresh stock across vendors, balances store-hopping delivery fees against product savings, and proves whether splitting the order is actually cheaper than buying everything from one convenient store.

2. **Lack of Reverse Bidding for Custom Requirements (RFQ):** In traditional offline commerce, when someone needs bulk groceries or specific quality grades, they talk directly to local merchants to negotiate prices. In existing digital platforms, communication is strictly one-way: the store sets a rigid price, and the customer either pays it or leaves. There is no platform where a customer can post a requirement in natural language and receive itemized line-item quotations from multiple nearby shops.

3. **Real-Time Synchronization and Support Gaps:** In quick-commerce, customers want real-time visibility into order status changes without constantly refreshing the browser. In addition, when common order issues arise (such as delivery delays or cancellations), human customer support is often slow, while rule-based chatbots fail to understand context or handle sensitive user data securely.

1.2 Objectives of the Project

The key objectives of this project are:

1. To build an **Autonomous AI Grocery Shopping Agent** that takes user requests in natural language or Hinglish, understands the household size, duration, and budget, queries multiple local stores, verifies available inventory, and calculates the optimal basket while factoring in individual store delivery charges.

2. To implement a visual **Store Journey Audit** that shows the customer the exact multi-store picking route and clearly presents the net cost savings compared to single-store convenience.

3. To build a **Merchant Quotation Engine (Module B)** supporting a reverse Request for Quotation (RFQ) marketplace where:

    - Customers can submit shopping requirements with target budgets and delivery deadlines.

- Nearby merchants can view open requests and submit itemized line-item quotations with individual unit prices, delivery charges, and stock confirmations.
- The platform dynamically ranks competing bids based on the customer's selected priority: Lowest Price, Fastest Delivery, Highest Rated Merchant, or Balanced Value.
- Customers and merchants can negotiate through counter-offers and quote revisions, with accepted quotations converting directly into confirmed orders.

4. To integrate a **Hybrid Recommendation System** using collaborative filtering, content feature vectors, and context-aware time-of-day scoring to personalize the customer's shopping feed.
5. To implement an **Automated Demand Prediction Engine** using time-series smoothing and linear regression running on an hourly cron job to provide stocking recommendations to store owners.
6. To develop a **Customer Support CRM** with automated intent classification, confidence scoring, real-time ticket messaging, and sensitive data (PII) masking.
7. To deploy the full-stack system to a cloud environment with a live database and verify its functionality through end-to-end automated tests.

1.3 Scope of the Project

The scope of this project encompasses:

- A single-page web application frontend built using React 18, utilizing CSS modules and custom dark-mode design tokens for clear readability and responsive usage across desktops and mobile browsers.
- A RESTful backend API built on Node.js and Express, integrated with MongoDB Atlas via Mongoose ODM for persistent storage of users, products, stores, orders, RFQ requests, quotations, and support tickets.
- A real-time bidirectional communication layer using Socket.IO for live order tracking, merchant RFQ alerts, and customer negotiation updates.
- An autonomous agent backend written in Python using LangGraph and FastAPI, interacting with local store databases and external retailer adapters.
- Role-based access control supporting both customer and store-owner accounts with protected routes and secure JWT authentication.

Items outside the immediate scope of this academic prototype include native mobile applications (iOS/Android APKs) and direct bank-integrated payment gateways. All monetary transactions in the prototype are simulated or use Cash on Delivery (COD) workflows.

1.4 Regional Market Context and Hyper-Local Economics

The retail grocery ecosystem in India presents distinct economic dynamics that fundamentally differ from Western retail markets. In tier-2 and tier-3 cities, as well as institutional campus environments (such as universities and hostels in Imphal, Manipur), household grocery purchases do not conform to the continuous micro-ordering pattern promoted by quick-commerce startups. Instead, consumer purchasing behavior is characterized by distinct structural factors:

1. **Significant Price Dispersion Across Neighborhood Markets:** Due to fragmented wholesale supply chains and variable transport overheads, identical staple commodities exhibit price variations of 12% to 25% between competing independent stores separated by less than two kilometers. For instance, specialized grain merchants at local wholesale mandis frequently provide substantial discounts on 20 kg or 25 kg sacks of rice and pulses, while neighborhood corner shops maintain higher markups to cover convenience overheads. A platform that enables digital comparison and competitive bidding allows households and institutional buyers to capture these savings directly.

2. **Economic Centrality of Independent Kirana Stores:** Independent neighborhood merchants (kirana stores) account for over 80% of retail grocery distribution across India. These merchants possess deep localized knowledge of regional consumption patterns, maintain agile relationships with agricultural distributors, and frequently operate on tight operational margins. However, centralized quick-commerce platforms bypass these merchants entirely by setting up proprietary dark store networks, displacing local businesses. GroDealz is specifically designed as an open, multi-tenant marketplace where neighborhood stores can participate digitally without paying extortionate commission rates.

3. **High Price Sensitivity on Bulk Staples:** While individual consumers may tolerate higher prices for occasional emergency purchases (such as bread or batteries delivered in ten minutes), routine grocery shopping involving 15 to 30 items represents a significant portion of a middle-class household or student mess budget. In bulk purchasing scenarios, a net difference of Rs. 250 to Rs. 400 across a monthly basket justifies waiting several hours or choosing scheduled delivery over ultra-fast 10-minute dispatch.

4. **Bargaining and Custom Requirement Culture:** Unlike fixed-price retail, offline Indian retail incorporates negotiation and relationship-driven pricing for recurring or large-scale orders. Buyers expect merchants to quote competitive prices when purchasing in bulk. The reverse Request for Quotation (RFQ) mechanism implemented in GroDealz directly digitizes this traditional commercial behavior, providing a transparent, auditable channel for price discovery.

1.5 Project Development Methodology and Iterative Lifecycle

To ensure systematic development and rigorous validation, GroDealz was engineered following an Agile, sprint-driven methodology organized into five distinct two-week milestones:

- **Milestone 1 — Foundations and Multi-Store Catalog:** Designed the core 3-tier system architecture, MongoDB schemas (User, Store, Product, Order), Express REST endpoints, and the React 18 user interface. Implemented initial catalog browsing, category filtering, and authentication with JWT tokens and bcrypt password encryption.
- **Milestone 2 — Algorithmic Recommendation and Demand Forecasting:** Formulated and integrated the hybrid recommendation engine combining collaborative filtering, content feature vectors, and context-aware time-of-day scoring. Implemented the background automated demand forecasting cron job using time-series smoothing and linear trend regression.
- **Milestone 3 — Autonomous AI Shopping Agent (LangGraph):** Constructed the cyclic state-machine decision graph in Python using LangGraph and FastAPI. Implemented multi-store catalog parallel querying adapters, inventory freshness checks, and the combinatorial basket optimizer that balances product price differentials against store-specific delivery fees. Built the interactive Store Journey Audit UI.
- **Milestone 4 — Module B Reverse RFQ Marketplace and Quotation Engine:** Developed the reverse shopping requirement workflow. Built the NLP requirement parser for extracting commodities, quantities, budgets, and deadlines. Implemented the merchant itemized bidding dashboard, four dynamic multi-criteria quotation ranking modes, counter-offer state management, and atomic conversion to confirmed orders.
- **Milestone 5 — Real-Time Synchronization, Support CRM, and Verification:** Implemented WebSocket event multiplexing via Socket.IO for instantaneous order tracking and RFQ room alerts. Developed the customer support CRM with automated intent classification and sensitive PII masking. Executed automated end-to-end integration test suites across all core subsystems.

1.6 Structure of the Report

The remainder of this report is organized as follows:

- **Chapter 2 (Literature Survey):** Reviews previous work in recommendation algorithms, e-grocery supply chains, demand forecasting techniques, autonomous agent frameworks, and reverse auction mechanisms.
- **Chapter 3 (Requirement Engineering):** Details the functional and non-functional requirements, stakeholder roles, and core use cases.

- **Chapter 4 (System Design):** Explains the architectural design, database schemas, LangGraph agent workflow, quotation ranking mathematics, and real-time event flow.
- **Chapter 5 (Implementation):** Describes the technical implementation, detailing the core services, algorithms, and frontend component structures.
- **Chapter 6 (Results and Interface Walkthrough):** Presents the test results, performance metrics, and a detailed screenshot walkthrough of the functioning website.
- **Chapter 7 (Conclusion and Future Scope):** Summarizes the completed work, discusses limitations, and outlines future enhancements.

Chapter 2

Literature Survey

This chapter reviews relevant academic literature, commercial systems, and existing technical approaches related to online grocery platforms, recommendation systems, demand forecasting, autonomous agents, and reverse auction marketplaces.

2.1 Evolution of Online Grocery and Quick-Commerce

In recent years, the online grocery market has shifted from scheduled next-day deliveries (such as traditional BigBasket or Amazon Fresh) toward hyper-local quick-commerce platforms like Blinkit, Zepto, and Swiggy Instamart. These platforms rely on geographically distributed micro-warehouses called "dark stores," each holding between two thousand and four thousand common stock-keeping units (SKUs) located within a three-kilometer radius of urban neighborhoods.

While this model achieves very low delivery times, studies on retail logistics note several structural constraints:

- **Inventory Limitations:** A single dark store cannot maintain the variety of items found across an entire city or district. When niche brands, bulk bags, or seasonal local produce are needed, the customer faces out-of-stock messages.
- **Pricing Inflexibility:** Prices in centralized dark stores are determined by algorithmic markup rules with no room for buyer negotiation or local merchant competition.
- **Exclusion of Local Neighborhood Shops:** Independent local merchants, who often possess superior connections with regional farmers and wholesale grain markets, are largely excluded from modern quick-commerce infrastructure.

2.2 Recommendation Systems in Retail

Personalized product recommendation is widely recognized as a primary driver of customer retention in e-commerce.

2.2.1 Collaborative Filtering

Collaborative filtering (CF), introduced formally by Resnick et al. [7] and expanded by Sarwar et al. [8], predicts user preferences by identifying patterns among similar users or items. In item-based CF, the similarity between two items i and j is computed across the user rating vector using the cosine similarity metric:

$$\text{sim}(i,j) = \frac{\sum_{u \in U} R_{u,i} \cdot R_{u,j}}{\sqrt{\sum_{u \in U} R_{u,i}^2} \cdot \sqrt{\sum_{u \in U} R_{u,j}^2}} \tag{2.1}$$

While collaborative filtering works well when substantial interaction data is available, it suffers from the well-documented *cold-start problem*: when a new product is added or a new customer signs up, there are no prior interactions, causing collaborative filtering alone to fail.

2.2.2 Content-Based Filtering

Content-based filtering, discussed by Pazzani and Billsus [6], avoids the cold-start problem by analyzing product features directly (such as category, brand, dietary attributes, and price tier). A user profile vector is constructed from the attributes of items the user has previously interacted with, and candidate products are scored by vector similarity. However, content-based methods tend to suffer from *overspecialization*, repeatedly recommending items very similar to what the user already purchased.

2.2.3 Hybrid Approaches

Burke [3] showed that combining collaborative and content-based approaches in an ensemble overcomes the individual weaknesses of both methods. In GroDealz, we implement a weighted hybrid ensemble that also incorporates contextual signals, such as the time of day (e.g., boosting milk and bakery products in the morning, and dinner vegetables and rice in the evening).

2.3 Demand Forecasting in Perishable Retail

Accurate demand prediction is essential for grocery stores because fresh items like vegetables, fruits, and dairy have very short shelf lives. Overstocking causes food spoilage and financial loss, while understocking leads to lost sales and disappointed customers.

Classic time-series methods such as the Box-Jenkins ARIMA models [2] require stationary data and significant computational overhead. For hyper-local stores operating on standard web servers, simpler lightweight techniques are often more practical:

- **Exponential Smoothing:** Gives exponentially decreasing weights to older observations, allowing recent order bursts to adjust the forecast quickly.
- **Moving Average Trend:** Tracks baseline demand over a multi-day rolling window to capture underlying weekly cycles.

- **Multivariate Linear Regression:** Relates order volume directly to calendar features such as hour of the day, day of the week, and weekend indicators.

Armstrong [1] established that combining separate forecasting models into an ensemble frequently outperforms individual models. GroDealz adopts this insight by averaging exponential smoothing with linear regression to generate hourly per-product demand forecasts.

2.4 Autonomous Agents and Multi-Step Task Execution

Recent breakthroughs in Large Language Models (LLMs) and agentic frameworks have fundamentally altered automated decision systems in e-commerce. Early conversational systems relied either on scripted decision trees (Weizenbaum [9]) or intent-classification chatbots with rigid slot-filling (Følstad and Brandtzæg [5]). While effective for narrow FAQs, such systems cannot perform multi-step planning, stateful backtracking, or dynamic tool execution.

The introduction of the ReAct paradigm by Yao et al. [10] demonstrated that prompting language models to alternate between explicit reasoning traces ("Thought") and executable environment actions ("Act") significantly improves performance on complex multi-hop queries. In hyper-local grocery shopping, an agent cannot simply answer a question; it must execute an autonomous operational loop:

1. **Structured Parameter Extraction:** Parse informal colloquial natural language or Hinglish into validated domain entities (family size N, duration D, target budget B, dietary tags).

2. **Parallel Environment Discovery:** Asynchronously query multiple distributed merchant databases to retrieve live product candidates, prices, and stock indicators.

3. **Data Normalization and Constraint Verification:** Standardize disparate unit measurements (e.g., converting 500 g packets and 1 kg sacks into unified unit-cost metrics) and enforce hard budget constraints.

4. **Combinatorial Basket Optimization:** Formulate and solve the single-store versus multi-store optimization tradeoff, balancing per-item price savings against individual store delivery charges.

5. **Auditable Itinerary Generation:** Compile an inspectable, human-verifiable store picking audit that allows the buyer to review individual store contributions before order placement.

Traditional agent frameworks like LangChain standard chains execute purely linear directed acyclic graphs (DAGs), which fail when an agent encounters unexpected real-world edge cases (such as a chosen item being out of stock across all preferred stores). To support iterative refinement and conditional recovery, GroDealz employs **LangGraph**, a cyclic state-machine framework where agent execution is modeled as

a state graph with typed state channels, conditional branching, and deterministic fallback nodes. By constraining language model outputs to strictly typed Pydantic data schemas, the system eliminates mathematical hallucination, ensuring that all item quantities and financial figures reflect verified database records.

2.5 Reverse Auctions and Multi-Attribute Request for Quotation (RFQ) Systems

In standard forward e-commerce marketplaces (such as Amazon or Blinkit), price discovery is strictly one-directional: the vendor sets a take-it-or-leave-it price, and the customer has no channel to express willingness-to-pay or request customized volume discounts. In contrast, reverse auction mechanisms invert this market dynamic: the buyer announces a specific requirement, and multiple pre-qualified suppliers submit competitive bids to fulfill the order.

Extensive literature on auction theory (De Vries and Vohra [4]) distinguishes between several reverse bidding formats:

- **First-Price Sealed-Bid Reverse Auctions:** Suppliers submit independent, private bids without visibility into competing offers. The buyer awards the contract to the lowest bidder. While computationally simple, sealed-bid formats often induce the "winner's curse" and prevent suppliers from dynamically adjusting prices based on real-time capacity.
- **Iterative English Reverse Auctions:** Suppliers openly observe competing bids and progressively lower their prices in descending rounds. While driving prices down to marginal costs, open descending auctions can trigger predatory bidding wars and discourage small merchant participation.
- **Multi-Attribute Interactive Quotation Negotiation:** In consumer and institutional procurement, price is rarely the sole evaluation criterion. Delivery reliability, fulfillment lead time, merchant reputation, and itemized product brand quality are equally vital. Multi-Attribute Utility Theory (MAUT) provides a mathematical basis for evaluating multi-dimensional bids by constructing a composite scoring function:

$$U(\vec{x}) = \sum_{k=1}^{K} w_k \cdot v_k(x_k) \tag{2.2}$$

  where w_k denotes the normalized importance weight of attribute k, and $v_k(x_k)$ is the attribute-specific value function.

GroDealz implements an itemized multi-attribute quotation engine (Module B). Rather than bidding arbitrary lump sums, merchants submit granular line-item prices alongside explicit delivery timestamps and stock commitments. Furthermore, the platform empowers the customer to choose their immediate evaluation priority dynamically—prioritizing lowest cost, fastest delivery speed, highest vendor reputation, or a balanced 100-point composite score. This guarantees transparent price formation while preventing merchants from concealing inflated delivery or packaging fees inside opaque bundles.

2.6 Data Security, Privacy, and Transactional Concurrency in Multi-Sided Platforms

Multi-sided digital marketplaces handle concurrent interactions between competing merchants and buyers, introducing critical data protection and concurrency challenges:

1. **Personally Identifiable Information (PII) Protection in Customer Support:** When buyers submit support tickets or interact with automated AI CRM systems regarding delivery disputes, they frequently share sensitive details such as contact telephone numbers, home delivery landmarks, and transaction identifiers. To comply with modern data privacy principles, sensitive fields must be sanitized before being transmitted to external language model APIs or centralized analytics logs. Implementing rule-based regex masking on the client and server boundary ensures that phone numbers and email addresses are replaced with anonymized tokens (e.g., `[PHONE_REDACTED]`) while preserving the semantic context required for intent classification.

2. **Concurrency and Atomic State Transitions in Reverse RFQs:** In an active reverse marketplace, multiple events occur asynchronously: merchants submit bids, buyers issue counter-offers, and quotations approach expiration timestamps. A critical concurrency hazard arises when a customer accepts a quotation: the selected quote must transition to `accepted`, all other active quotations for that request must be atomically updated to `rejected`, a confirmed `Order` document must be instantiated, and the parent `BuyerRequest` status must become `converted`. If these operations are executed non-atomically, race conditions could allow a merchant to revise a quote simultaneously or enable double-acceptance. Enforcing atomic database document updates and transactional validation prevents state inconsistency.

3. **Role-Based Cryptographic Access Control:** Because merchants and customers share the same underlying API endpoints, the application server must cryptographically verify user identity and authority on every request. JSON Web Tokens (JWT) signed with high-entropy HMAC-SHA256 secrets provide stateless, tamper-proof session tokens containing encrypted user identifiers and operational roles, preventing unauthorized access across disparate merchant dashboards.

2.7 Comparative Summary

Table 2.1 highlights how GroDealz compares with standard existing platforms across essential functional areas.

Table 2.1: *Comparison of GroDealz with Existing Grocery Delivery Approaches*

Feature	**Quick Commerce**	**Traditional Grocery**	**GroDealz**
Store Model	Single Dark Store	Central Warehouse	Multi-Store Local Network
Autonomous AI Planner	None (Search Only)	None (Search Only)	Natural Language Multi-Store Agent
Basket Optimization	No Comparison	Single Basket	Multi-Store vs Single-Store Net Savings
Reverse RFQ Marketplace	Not Supported	Not Supported	Full Itemized Merchant Bidding (Module B)
Bid Ranking Modes	Fixed Price	Fixed Price	Lowest Price, Fastest Delivery, Rating, Balanced
Live Store Journey Audit	None	None	Visual Picking Audit and Savings Proof
Real-Time Tracking	Polling / SMS	Delivery Window	Live WebSockets (Socket.IO)

Chapter 3

Requirement Engineering

This chapter presents the requirements analysis for the GroDealz platform. It defines the target user personas, lists all functional requirements (FRs) categorized by subsystem, outlines key non-functional requirements (NFRs), and details practical use case scenarios.

3.1 Stakeholder Analysis

The system interacts with three primary groups of users, each having distinct motivations:

1. **Customer (Consumer):** Wants to purchase groceries quickly and at the best possible price. They may want to search for individual items, generate an optimized multi-store basket automatically from a plain text meal request, or post custom bulk requirements (such as 25 kg of rice) and review competitive merchant bids.
2. **Store Owner (Merchant):** Owns an independent local store or supermarket. They want to list their inventory, monitor real-time orders, browse nearby customer shopping requests, submit itemized quotations with custom delivery fees, and view automated demand forecasts to keep popular items in stock.
3. **Platform Administrator:** Oversees platform health, manages user roles, verifies store profiles, and ensures catalog integrity.

3.2 Functional Requirements

The functional requirements are organized systematically by subsystem:

3.2.1 Authentication and User Profiles (FR-AUTH)

- **FR-AUTH-1:** The system shall allow users to register with name, email, phone number, address, and an initial role (customer or store owner).

- **FR-AUTH-2:** The system shall authenticate users using hashed passwords (via `bcrypt`) and issue stateless JSON Web Tokens (JWT) with a configured expiration time.
- **FR-AUTH-3:** Customers shall be able to update their dietary preferences (e.g., vegetarian, vegan, non-vegetarian, gluten-free) to inform recommendation filters.

3.2.2 Product Catalog and Discovery (FR-CATALOG)

- **FR-CATALOG-1:** Customers shall be able to browse products by category (Vegetables, Fruits, Dairy, Staples, Snacks, Bakery, Beverages, Meat & Eggs).
- **FR-CATALOG-2:** The system shall provide real-time search with instant filtering by price range, discount level, and dietary tag.
- **FR-CATALOG-3:** The homepage shall present personalized recommendation rails generated by a hybrid recommendation algorithm combining user interaction history, content vectors, and time-of-day affinity.

3.2.3 Autonomous AI Shopping Agent and Multi-Store Optimizer (FR-AGENT)

- **FR-AGENT-1:** The system shall accept grocery shopping requests in natural language or Hinglish (e.g., *"Buy groceries for a family of four for three days under Rs. 1,500"*).
- **FR-AGENT-2:** The agent pipeline shall extract structured parameters including household members, duration, target budget, delivery urgency, and dietary goals.
- **FR-AGENT-3:** The agent shall query multiple nearby participating stores, verify live stock availability, and evaluate whether a single-store order or a multi-store split provides the lowest overall cost after including individual delivery fees.
- **FR-AGENT-4:** The system shall display an interactive Store Journey Audit showing the customer the exact store-hopping route and proving the net financial savings.

3.2.4 Reverse RFQ Marketplace and Merchant Quotation Engine (FR-RFQ)

- **FR-RFQ-1:** Customers shall be able to post custom shopping requirements by entering natural language descriptions or specifying structured line items.
- **FR-RFQ-2:** The system shall parse prompts to extract individual items, quantities, units (kg, g, L, packs), target budget, and delivery deadlines.

- **FR-RFQ-3:** Customers shall have the option to share their target budget with merchants or keep it private.
- **FR-RFQ-4:** Nearby merchants shall be able to view open buyer requests in their area and submit itemized line-item quotations specifying unit prices, brand offered, stock confirmation, delivery fees, taxes, and expiration times.
- **FR-RFQ-5:** The quotation engine shall rank competing quotations dynamically according to the customer's choice: *Lowest Total Price*, *Fastest Delivery*, *Top Rated Merchant*, or *Balanced Composite Value*.
- **FR-RFQ-6:** Customers shall be able to propose counter-offers with notes, and merchants shall be able to revise their quotations accordingly.
- **FR-RFQ-7:** Accepting a quotation shall atomically mark competing bids as rejected and convert the accepted quotation directly into a confirmed order.

3.2.5 Order Lifecycle and Real-Time Tracking (FR-ORDER)

- **FR-ORDER-1:** Customers shall be able to add items to a persistent shopping cart and proceed through checkout with address confirmation and payment selection.
- **FR-ORDER-2:** The system shall update order status across six defined states: *placed, confirmed, preparing, out_for_delivery, delivered, cancelled*.
- **FR-ORDER-3:** Real-time order progress updates shall be transmitted to the customer and store owner via Socket.IO without requiring page reloads.

3.2.6 Demand Prediction and Store Operations (FR-DEMAND)

- **FR-DEMAND-1:** An automated background cron job shall execute every hour to compute smoothed hourly demand forecasts for each product SKU.
- **FR-DEMAND-2:** The store dashboard shall present actionable operational alerts regarding inventory replenishment and staffing requirements during peak hours.

3.2.7 AI Customer Support CRM (FR-SUPPORT)

- **FR-SUPPORT-1:** Customers shall be able to create support tickets with optional sensitive data (PII) masking for payment card numbers and contact details.
- **FR-SUPPORT-2:** The system shall automatically classify ticket intents (e.g., order cancellation, delivery delay, general query) and compute an AI confidence score.
- **FR-SUPPORT-3:** A real-time bidirectional chat interface shall allow communication between customers and support staff.

3.3 Non-Functional Requirements

- **Performance (NFR-1):** Catalog browsing and filtering API requests shall respond within 100 ms under normal load. WebSocket status updates shall have a network propagation latency of less than 50 ms on local connections.
- **Security (NFR-2):** User passwords must be hashed using `bcrypt` with at least 10 salt rounds. Protected API endpoints must validate cryptographic JWT signatures. Payment details and sensitive numbers in support tickets must be sanitised before persisting to database storage.
- **Data Integrity and Idempotency (NFR-3):** Order conversion from quotations and checkout operations must include idempotency keys to prevent duplicate order generation from rapid clicks or retries.
- **Usability and Accessibility (NFR-4):** The web application must provide a responsive, dark-mode user interface that maintains clear contrast, readable typography, and accessible touch targets across desktop and mobile screens.

3.4 Core Use Case Scenarios

3.4.1 Use Case 1: Autonomous Multi-Store Grocery Basket

Actor: Customer
Preconditions: Customer is logged in and navigates to the AI Copilot page.
Flow of Events:

1. Customer types: *"Party snacks and beverages for 5 people under Rs. 600"*.
2. The system extracts parameters: 5 members, 3 days duration, Rs. 600 target budget, snacks and beverages categories.
3. The agent inspects inventory across FreshMart Central and QuickBazaar.
4. The optimizer detects that snacks are cheaper at QuickBazaar while beverages are cheaper at FreshMart.
5. The system calculates that splitting the order saves Rs. 267 compared to buying everything from one store, even after adding delivery fees.
6. The customer reviews the picking audit and places the order with one click.

3.4.2 Use Case 2: Custom Requirement Reverse RFQ and Merchant Quotation

Actor: Customer and Multiple Nearby Merchants
Preconditions: Customer needs bulk goods not listed as standard SKUs.
Flow of Events:

1. Customer enters: *"I need 25 kg basmati rice for a hostel. My budget is Rs. 2,500. Delivery within two days."* on the Custom RFQ page.
2. The system parses the text, sets budget to Rs. 2,500, delivery deadline to 2 days, and creates an open RFQ.
3. A real-time notification alerts nearby merchants on their Open Requests dashboard.
4. Shop A quotes Rs. 94/kg (Rs. 2,350) + Rs. 50 delivery = Rs. 2,400.
5. Shop B quotes Rs. 90/kg (Rs. 2,250) + Rs. 100 delivery = Rs. 2,350.
6. Shop C quotes Rs. 91.20/kg (Rs. 2,280) + free delivery = Rs. 2,280.
7. The customer views the Offer Comparison Matrix. When sorted by lowest price, Shop C ranks #1.
8. The customer clicks "Accept Offer & Order." The quote converts into an order, and competing bids are closed.

Chapter 4

System Design

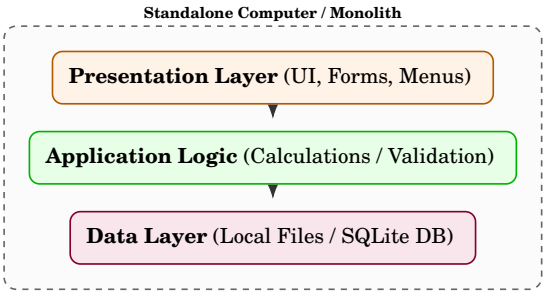
This chapter translates the requirements specified in Chapter 3 into architectural, database, mathematical, and algorithmic designs. It details the high-level system tiers, data models, the autonomous agent workflow, the merchant quotation engine formulas, and real-time communication events.

4.1 Architectural Evolution: 1-Tier, 2-Tier, and 3-Tier Models

Before examining the detailed design of GroDealz, it is useful to review the architectural evolution from monolithic 1-tier and 2-tier client-server designs to the modern 3-tier architecture.

4.1.1 1-Tier vs. 2-Tier Architectural Models

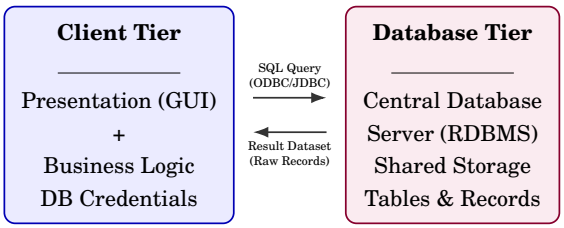
Figure 4.1 illustrates the fundamental differences between 1-tier and 2-tier architectures:



1-Tier Architecture:

All presentation interface, application logic, and database storage reside on a single standalone machine. It cannot support network collaboration, shared merchant inventory, or multi-buyer ordering.

(a) *1-Tier Monolithic Model (Single Standalone Machine)*



2-Tier Architecture:

Fat client machines connect directly over a local network to a central database server. It exposes database credentials inside the client binary and requires client upgrades whenever business logic changes.

(b) *2-Tier Client-Server Model (Direct Database Connection)*

Figure 4.1: *Comparison of 1-Tier and 2-Tier Architectural Models with Functional Descriptions*

1. **1-Tier Architecture (Monolithic):** In a 1-tier architecture, the presentation layer, application business logic, and database storage reside on the exact same physical computer (for example, a traditional desktop application using an embedded SQLite or Microsoft Access database). While this model requires zero network overhead, it is completely unsuitable for a multi-store grocery platform: multiple customers, store owners, and delivery riders cannot coordinate shared inventory, live orders, or real-time bids on a standalone machine.

2. **2-Tier Architecture (Traditional Client-Server):** In a 2-tier architecture, the presentation interface and application business logic run together on a "fat client" machine, which communicates directly over a local network or VPN to a centralized database server using database connection protocols (such as ODBC or JDBC). While this enables shared data storage, it introduces critical vulnerabilities:

    - *Security Risk:* Embedding direct database credentials within the client application exposes the core database to tampering.
    - *Tight Coupling and Maintenance Burden:* Any modification to business logic, pricing rules, or quotation ranking algorithms requires updating the client software on every user's device.
    - *Poor Scalability for AI Tasks:* Complex computations—such as cross-store basket optimization and demand forecasting—would consume local client device memory and cannot easily leverage server-side parallelism.

4.1.2 3-Tier Architecture of GroDealz

To overcome these structural limitations, GroDealz implements a clean **3-Tier Architecture**, decoupling the Presentation Tier, the Application and AI Logic Tier, and the Data Persistence Tier. Figure 4.2 depicts the architectural tiers and the communication flows between them.

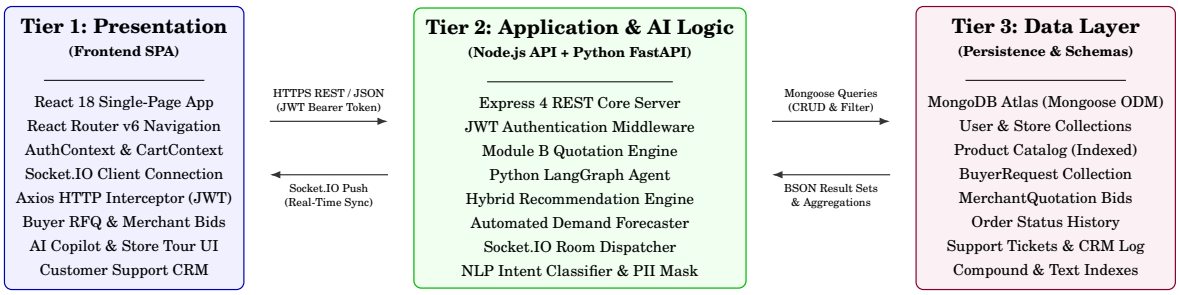


Figure 4.2: *3-Tier High-Level Architecture of the GroDealz Platform. Tier 1 manages customer interactions in React 18, Tier 2 processes quotation algorithms and AI agents in Node.js / Python, and Tier 3 secures relational and document data in MongoDB Atlas.*

The responsibilities of each tier are defined as follows:

1. **Tier 1 — Presentation Tier (Client):** Built as a lightweight, reactive Single-Page Application (SPA) using React 18. The presentation tier manages local state (such as the cart, user login status, active filters, and navigation paths) and renders the dark-mode responsive interface. It communicates with Tier 2 exclusively over secure HTTPS REST endpoints and persistent WebSocket channels, ensuring that no database credentials or internal business formulas are exposed to the client.

2. **Tier 2 — Application and AI Logic Tier (Server):** Implemented using Node.js with Express 4 and Python 3.10 with FastAPI/LangGraph. This tier acts as the centralized brain of the platform:

   - *Security & Authentication:* Validates cryptographic JWT signatures on incoming requests and enforces role-based authorization (customer vs. store owner vs. admin).
   - *Quotation Engine (Module B):* Evaluates line-item mathematics, computes dynamic ranking scores across customer-selected criteria, manages counter-offers, and coordinates atomic order conversion.
   - *Autonomous Shopping Agent:* Executes LangGraph decision graphs to search cross-store catalogs, verify inventory, optimize multi-store baskets, and compile picking itineraries.
   - *Real-Time Event Dispatcher:* Manages Socket.IO connection rooms (`merchants_rfq`, `rfq_<id>`, `order_<id>`) to broadcast instantaneous updates without client polling.

3. **Tier 3 — Data Persistence Tier (Database):** Managed by MongoDB Atlas via the Mongoose Object Document Mapper (ODM). Tier 3 guarantees transactional

persistence, document validation, and high-performance querying through compound indexes on frequently queried fields. Direct database access is strictly isolated to Tier 2 through private database URIs.

4.2 Database Design and Data Models

The database schema connects seven primary collections, as illustrated in the Entity-Relationship Diagram in Figure 4.3.

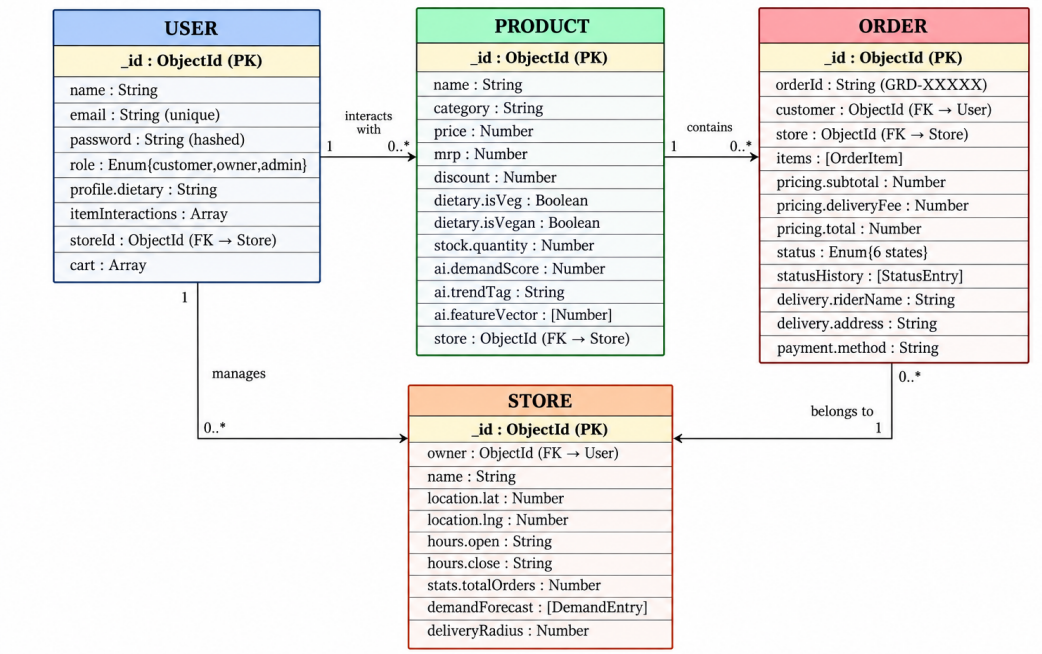


Figure 4.3: Entity-Relationship Diagram for GroDealz

Figure 4.3: *Entity-Relationship Diagram (ERD) of GroDealz Database. Illustrates relational dependencies across seven primary collections including User, Store, Product, Order, BuyerRequest, MerchantQuotation, and Customer Support.*

In addition to the standard User, Product, Store, and Order collections, GroDealz introduces two purpose-built schemas for the reverse marketplace:

4.2.1 Buyer Request Schema (`BuyerRequest`)

Stores custom customer shopping requirements:

- `requestId`: Unique identifier code (e.g., `RFQ-20261031-4821`).
- `customer`: Reference to the User ID of the buyer.
- `title`: Short requirement summary (e.g., “25 kg Basmati Rice for Hostel”).
- `rawPrompt`: Raw natural language prompt entered by the buyer.

- `items`: Structured array of requested items with quantities, units, and brand preferences.
- `budget`: Optional numerical target budget in Indian Rupees.
- `shareBudgetWithMerchants`: Boolean flag to share target budget with bidding merchants.
- `deliveryDeadline`: Required delivery completion timestamp.
- `rankingPriority`: Chosen sorting criterion (price, delivery, rating, or balanced).
- `status`: Current state (`open`, `quotes_received`, `negotiating`, `accepted`, `expired`).
- `convertedOrderId`: Reference to the created order when quotation is accepted.

4.2.2 Merchant Quotation Schema (`MerchantQuotation`)

Stores itemized quotations submitted by competing store owners:

- `quotationId`: Unique identifier code (e.g., `QUO-20261031-1049`).
- `request`: Reference to the parent `BuyerRequest`.
- `store` and `merchant`: References to the merchant store and user account.
- `lineItems`: Array of quoted line items with unit price, stock status, and line totals.
- `itemsSubtotal`, `deliveryCharge`, `tax`, `grandTotal`: Transparent cost breakdown.
- `stockConfirmed`: Boolean indicating immediate inventory reservation.
- `proposedDeliveryDate` and `expiresAt`: Proposed delivery date and offer expiry.
- `counterOffer`: Subdocument storing buyer counter-offer and negotiation note.
- `revisionHistory`: Historical log of updated quote versions.

Figure 4.4 shows the UML Class Diagram highlighting the relationship between these domain models and service engines.

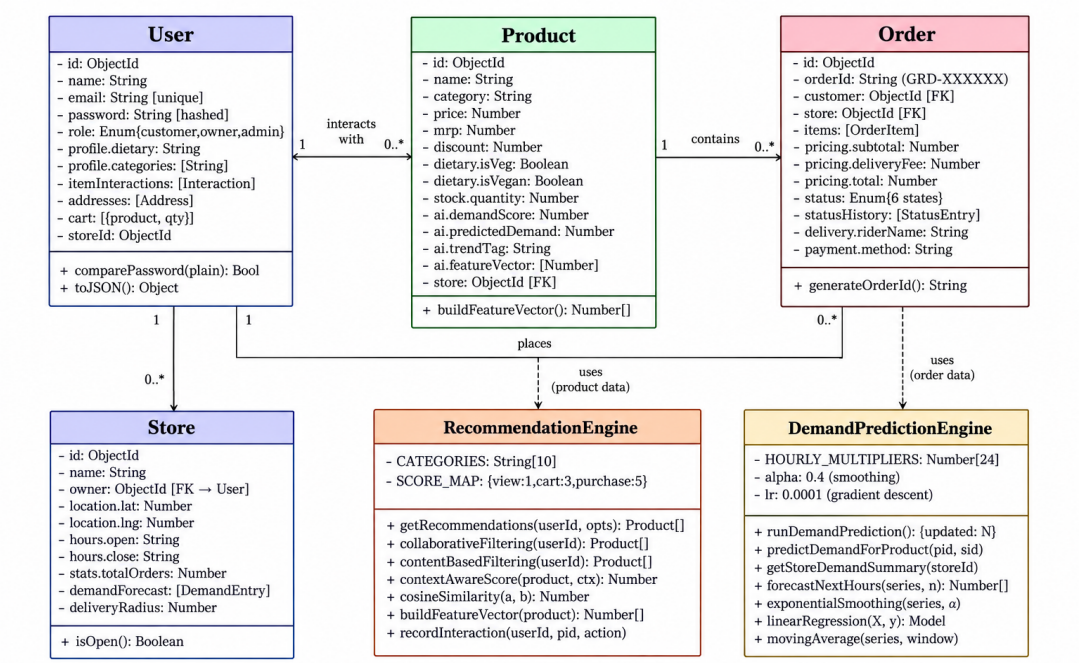


Figure 4.2: UML Class Diagram for GroDealz

Figure 4.4: *UML Class Diagram of System Architecture. Details domain model schemas, controller interfaces, algorithmic quotation engines, and real-time Socket.IO notification rooms.*

4.3 Autonomous AI Shopping Agent Workflow

The Autonomous AI Grocery Shopping Agent (accessible on `/ai-planner`) operates as a state-machine graph implemented via LangGraph. When a user submits an informal shopping goal (such as *"Party snacks and beverages for 5 people under Rs. 600"*), execution proceeds through five sequential nodes:

1. **Parameter Extraction Node:** Employs structured natural language parsing to extract the target count of individuals ($N = 5$), planning duration ($D = 3$ days), maximum target budget ($B = \text{Rs. } 600$), dietary constraints, and item categories.

2. **Cross-Store Discovery Node:** Queries participating local merchant databases (e.g., FreshMart Central, QuickBazaar, Paona Supermarket) to assemble candidate products matching each needed category.

3. **Fresh Stock Verification Node:** Verifies current shelf quantities and freshness indicators, filtering out items that are out of stock.

4. **Basket Optimization Node:** Solves the single-store versus multi-store tradeoff. Buying all items from one convenient store incurs a single delivery fee (F_{single}), whereas splitting items across multiple stores allows selecting the mini-

mum product price $P_{s,i}$ from store s at the cost of separate store delivery charges F_s:

$$C_{\text{single}} = \sum_{i \in I} P_{\text{single},i} + F_{\text{single}} \tag{4.1}$$

$$C_{\text{multi}} = \sum_{s \in S_{\text{visited}}} \left(\sum_{i \in I_s} P_{s,i} + F_s \right) \tag{4.2}$$

$$\Delta_{\text{savings}} = C_{\text{single}} - C_{\text{multi}} \tag{4.3}$$

If $\Delta_{\text{savings}} > 0$, the multi-store split is recommended to the customer as the best-value deal.

5. **Store Journey Audit Node:** Generates an audit trail detailing the exact items to pick at each shop and visualizes the itinerary in the user interface.

4.4 Module B Quotation Engine Design

The Merchant Quotation Engine governs the pricing, ranking, and negotiation lifecycle of custom shopping requirements.

4.4.1 Itemized Pricing Calculations

Merchants cannot simply quote an arbitrary lump sum; they must submit unit prices for every individual line item requested. This ensures transparency for the customer:

$$\text{lineTotal}_i = \text{quantity}_i \times \text{unitPrice}_i \tag{4.4}$$

$$\text{itemsSubtotal} = \sum_{i=1}^{n} \text{lineTotal}_i \tag{4.5}$$

$$\text{grandTotal} = \text{itemsSubtotal} + \text{deliveryCharge} + \text{tax} \tag{4.6}$$

4.4.2 Multi-Criteria Dynamic Ranking Formulations

Unlike standard platforms that sort solely by price, GroDealz allows customers to rank bids based on their immediate priority:

1. **Lowest Price Mode** (`lowest_price`)**:** Evaluates grand total cost. The lowest grand total receives the highest ranking score:

$$\text{Score}_{\text{price}} = 1{,}000{,}000 - \text{grandTotal} \tag{4.7}$$

The merchant with the minimum total is awarded the badge *"Lowest Total Price."*

2. **Fastest Delivery Mode** (`fastest_delivery`)**:** Evaluates the proposed delivery timestamp (t_{delivery}). Bids proposing earlier delivery receive higher scores:

$$\text{Score}_{\text{delivery}} = T_{\text{future}} - t_{\text{delivery}} \tag{4.8}$$

The earliest delivery bid receives the badge *"Fastest Delivery."*

3. **Top Rated Store Mode** (`highest_rated`)**:** Prioritizes merchant reliability based on customer reviews and ratings:

$$\text{Score}_{\text{rating}} = (\text{Rating} \times 1{,}000) + \min(\text{ReviewCount}, 100) \tag{4.9}$$

   The highest-rated vendor is highlighted with the badge *"Top Rated Store."*

4. **Balanced Composite Mode** (`balanced`)**:** Combines price, delivery speed, and store reputation into a normalized 100-point composite score:

$$\text{Score}_{\text{balanced}} = S_{\text{price}}(50 \text{ pts}) + S_{\text{speed}}(30 \text{ pts}) + S_{\text{reputation}}(20 \text{ pts}) \tag{4.10}$$

   where:

$$S_{\text{price}} = 50 \times \left(1 - \frac{\text{grandTotal} - \text{minPrice}}{\text{maxPrice} - \text{minPrice}}\right) \tag{4.11}$$

$$S_{\text{speed}} = 30 \times \left(1 - \frac{t_{\text{delivery}} - t_{\text{earliest}}}{t_{\text{latest}} - t_{\text{earliest}}}\right) \tag{4.12}$$

$$S_{\text{reputation}} = \left(\frac{\text{Rating}}{5.0}\right) \times 20 \tag{4.13}$$

   If the quoted total is within the customer's target budget, an additional 5-point bonus is applied.

4.5 Real-Time WebSocket Event Architecture

To provide instantaneous notifications without inefficient browser polling, GroDealz utilizes Socket.IO rooms:

- `join_rfq(requestId)`: Customer joins a private room for their request. When a merchant submits or revises a quote, the server emits `quote_update`.
- `join_merchants_rfq`: Merchants join a collective broadcast room. When a customer posts a new requirement, the server emits `new_rfq_available`.
- `track_order(orderId)`: Customers track real-time order status transitions (*placed* → *confirmed* → *delivered*).

Chapter 5

Implementation

This chapter describes the implementation details of the GroDealz platform. It outlines the technology stack, walks through the core backend services, and details the frontend component architecture.

5.1 Technology Stack

The implementation utilizes an integrated, multi-service architecture:

- **Frontend:** React 18 with functional components and hooks (`useState`, `useEffect`, `useContext`, `useMemo`). Routing is managed by React Router v6. Styling is built using custom CSS modules with dark-mode design tokens, ensuring responsive layouts without external heavyweight UI frameworks.
- **Backend Core (REST and Real-Time):** Node.js runtime with Express 4 web framework. Bidirectional communication is implemented using Socket.IO v4. Background scheduled tasks (such as hourly demand forecasts) are executed via `node-cron`. Security is enforced using `bcryptjs` for password hashing, `jsonwebtoken` for token creation and validation, `cors` for origin management, and `express-rate-limit` for request throttling.
- **Database and Persistence:** MongoDB Atlas cloud database managed via the Mongoose 7 Object Document Mapper (ODM). Indexes are defined on frequent query paths (such as `requestId`, `customer`, `store`, and `status`).
- **Autonomous AI Agent Service:** Built using Python 3.10 with FastAPI and LangGraph. It models multi-step reasoning nodes for cross-store catalog searching and basket cost optimization.

5.2 Core Backend Services

5.2.1 Natural Language Requirement Parser (`rfqParser.js`)

The requirement parser takes arbitrary customer sentences (such as *"I need 25 kg basmati rice for a hostel. My budget is Rs. 2,500. Delivery within two days."*) and extracts structured parameters using regular expressions and semantic fallback dictionaries:

- **Budget Extraction:** Strips currency symbols (Rs. , Rs, INR) and comma digit separators, matching numerical patterns adjacent to budget keywords:

```
const normalizedQuery = query.replace(/(\d),(\d)/g, '$1$2');
const budgetMatch = normalizedQuery.match(
  /(?:budget\s*(?:is|of)?\s*[:=]?\s*|under\s*|upto\s*)(?:rs\.?|
     inr|)?\s*(\d{2,7})/i
);
if (budgetMatch) {
  result.budget = parseInt(budgetMatch[1].replace(/,/g, ''), 10);
}
```

- **Delivery Deadline Extraction:** Scans for day counts (e.g., *"within 2 days"*, *"in 3 days"*) or relative terms (such as *"tomorrow"* or *"urgent"*) and computes the future `Date` timestamp.
- **Commodity and Quantity Extraction:** Matches patterns pairing numbers with measurement units (kg, g, L, packs, pcs) and product names (such as *"25 kg basmati rice"* or *"2 litre mustard oil"*).

5.2.2 Merchant Quotation Engine (`quotationEngine.js`)

The quotation engine implements four primary operations:

1. **Line-Item Validation and Calculation (`validateAndCalculateQuote`):** Ensures every quoted item has a non-negative unit price and valid quantity, calculating exact line totals, subtotal, and grand total:

```
const lineTotal = Math.round(qty * unitPrice * 100) / 100;
const itemsSubtotal = calculatedItems.reduce((acc, it) => acc +
   it.lineTotal, 0);
const grandTotal = itemsSubtotal + deliveryCharge + tax;
```

2. **Dynamic Multi-Criteria Ranking (`rankQuotations`):** Computes ranking scores according to the customer's selected priority (`lowest_price`, `fastest_delivery`, `highest_rated`, or `balanced`), assigning intuitive badges to top-ranking bids.
3. **Counter-Offer and Revision Handlers:** Manages negotiation state. When a customer counters, the quote is flagged as `customer_countered`. When a merchant revises, the previous version is archived in `revisionHistory` and new totals are calculated.

4. **Atomic Conversion to Order (`acceptQuotationAndCreateOrder`):** Verifies quote expiration, marks the selected quote as `accepted`, rejects competing bids, creates a confirmed `Order` document, and links the order ID to the original `BuyerRequest`.

5.2.3 Hybrid Recommendation Subsystem

The recommendation service in `backend/routes/recommendations.js` calculates scores using a three-tier model:

1. **Collaborative Filtering:** Computes cosine similarity between user interaction history vectors. If a user is new (cold-start), the engine falls back to trending items sorted by total order count.

2. **Content-Based Scoring:** Evaluates 16-dimensional feature vectors representing product categories, dietary tags, and price points.

3. **Context-Aware Scoring:** Analyzes local time of day to apply category multipliers (+0.10 for Dairy and Bakery in morning hours; +0.12 for Vegetables and Staples in evening hours).

5.2.4 Automated Demand Prediction

Implemented in `backend/ai/demandPrediction.js`, an automated `node-cron` job triggers hourly. It aggregates past order volumes over a rolling 168-hour window, applies single exponential smoothing ($\alpha = 0.4$), computes moving-average trend direction, and calculates multivariate regression to output predicted demand scores and staffing recommendations.

5.3 Frontend Architecture and Components

The user interface is organized into modular pages located in `frontend/src/pages/`:

5.3.1 Customer Shopping Requirements Page (`BuyerRFQ.js`)

- **Prompt Input with AI Extractor:** Features an interactive input box with a 1-click example loader (*"I need 25 kg basmati rice for a hostel..."*). Clicking *"Extract with AI Parser"* calls the backend API and auto-fills the line-item table.

- **Structured Line-Item Editor:** Allows users to add, edit, or delete items, specify units, and record preferred brands.

- **Offer Comparison Matrix:** Displays competing merchant cards side-by-side. Users can switch ranking priorities dynamically with one click, expand line-item breakdowns to view unit prices, propose counter-offers, or accept the winning quote to generate an immediate order.

5.3.2 Merchant Quotation Dashboard (`MerchantRFQ.js`)

- **Open Buyer Requests Feed:** Displays open nearby requests with item details, delivery location, deadline countdown, and budget visibility indicator.
- **Itemized Quotation Modal:** Pre-fills requested items so the merchant can enter individual unit prices, set delivery fees and taxes, confirm inventory stock, choose delivery dates, and set quote validity periods.
- **My Bids and Counter Negotiation View:** Highlights active bids. When a customer sends a counter-offer, an amber alert banner displays the proposed price and note, with a 1-click option to revise the quotation.

5.3.3 Autonomous AI Grocery Planner (`AIGroceryPlanner.js`)

Provides the conversational grocery interface with parameter chips (Family Size, Duration, Budget, Diet, Goal), a visual Store Journey Audit showing the picking route, and a comparative savings banner highlighting the multi-store net advantage over single-store purchases.

5.3.4 Customer Support CRM (`Support.js`)

Allows customers to create tickets with an optional PII masking toggle. It features automated intent classification, AI confidence percentage ratings, CRM tracking identifiers, and a live chat panel.

5.3.5 Homepage and Product Catalog (`Home.js` and `Products.js`)

Presents the full grocery catalog with category pills, live search, freshness badges, and personalized recommendation rails.

5.4 Autonomous LangGraph Decision Graph Implementation

The autonomous shopping assistant is implemented in the Python microservice (`ai-agent/app/graph.py`) using LangGraph. Unlike rigid conversational scripts, the agent maintains an explicit, typed state graph throughout its multi-step execution.

5.4.1 State Schema Specification (`AgentState`)

The workflow state is defined as a typed dictionary passing data between nodes:

```
class AgentState(TypedDict):
    raw_query: str                          # User natural language
        input
    requirements: Optional[Dict[str, Any]]  # Extracted parameters
```

```
    raw_candidates: List[Dict[str, Any]]     # Items discovered across
        stores
    normalized_candidates: List[Dict[str, Any]] # Standardized units &
        prices
    optimized_strategies: Dict[str, Any]     # Single vs Multi store
        metrics
    proposal: Dict[str, Any]                 # Audited picking
        itinerary
    status: str                              # Lifecycle stage
        indicator
    errors: List[str]                        # Resilience error log
```

5.4.2 Graph Node Pipeline and Execution Flow

The decision graph executes seven sequential operational nodes:

1. `extract_requirements_node`**:** Invokes an LLM prompt with strict Pydantic output parsing (`ShoppingRequirements`) to extract household size, planning duration, target budget, and item lists.

2. `validate_requirements_node`**:** Performs sanity validation. If the customer submits an open-ended goal (e.g., *"Groceries for 3 days"*) without specific items, it dynamically injects baseline staples (vegetables, rice, milk, dal, bread) scaled by family size.

3. `multi_store_search_node`**:** Iterates through needed items and calls `store_registry.parallel_search()`, executing concurrent asynchronous HTTP requests across all registered store adapters (FreshMart Central, QuickBazaar, Paona Supermarket).

4. `normalize_items_node`**:** Unifies unit representations across stores (e.g., converting grams to kilograms, and matching brand variations).

5. `optimize_basket_node`**:** Computes total costs for each individual store (C_{single}) and calculates the optimal cross-store partition (C_{multi}) by greedily picking the lowest price per item while adding delivery fees (F_s) for each distinct store visited.

6. `generate_proposal_node`**:** Assembles the final structured response containing the Store Journey Audit, itemized picking breakdown per vendor, and proof of net savings (Δ_{savings}).

The graph edges are configured using LangGraph's `StateGraph`:

```
workflow = StateGraph(AgentState)
workflow.add_node("extract", extract_requirements_node)
workflow.add_node("validate", validate_requirements_node)
workflow.add_node("search", multi_store_search_node)
workflow.add_node("normalize", normalize_items_node)
workflow.add_node("optimize", optimize_basket_node)
workflow.add_node("propose", generate_proposal_node)

workflow.set_entry_point("extract")
```

```
workflow.add_edge("extract", "validate")
workflow.add_edge("validate", "search")
workflow.add_edge("search", "normalize")
workflow.add_edge("normalize", "optimize")
workflow.add_edge("optimize", "propose")
workflow.add_edge("propose", END)
compiled_agent = workflow.compile()
```

5.5 Transaction Management and Atomic Quotation Acceptance

To prevent race conditions during reverse RFQ negotiations, `acceptQuotationAndCreateOrder` in `backend/services/quotationEngine.js` executes an atomic multi-document state transition:

1. **Expiration and Status Verification:** Verifies that the quotation has not passed its expiration timestamp (`Date.now() <= quote.expiresAt`) and that the parent request remains in an actionable state (`open` or `quotes_received`).
2. **Atomic Status Mutation:** Transitions the accepted quotation status to `accepted`. Concurrently, all other quotations linked to the same `requestId` are updated to `rejected`:

```
await MerchantQuotation.updateMany(
  { request: requestId, _id: { $ne: quotationId } },
  { $set: { status: 'rejected' } }
);
```

3. **Order Document Generation:** Instantiates a confirmed `Order` document containing an immutable snapshot of all quoted line items, agreed unit prices, delivery fee, tax, merchant store ID, and buyer customer ID.
4. **Parent Request Linking:** Updates `BuyerRequest` status to `accepted` and records the generated `orderId` in `convertedOrderId`.
5. **Real-Time Event Dispatch:** Dispatches WebSocket notifications to both the customer room and the winning merchant, alerting them that the transaction has been finalized.

5.6 Real-Time WebSocket Event Multiplexing

The real-time layer is managed in `backend/server.js` using Socket.IO. To prevent event cross-talk between unrelated users, connections are partitioned into isolated rooms:

- **Merchant RFQ Room** (`merchants_rfq`)**:** All authenticated merchants join this broadcast channel upon login. When any buyer posts a new requirement, the server emits `new_rfq_available` with summary details (delivery locality, item count, target deadline).

- **Request-Specific Negotiation Room** (`rfq_<id>`)**:** The buyer and participating merchants join a scoped room for that specific RFQ. When a merchant submits or revises an itemized quotation, or when the buyer submits a counter-offer, the server emits `quote_update` exclusively to room participants.
- **Live Order Tracking Room** (`order_<id>`)**:** When an order is placed, both the customer and fulfilling merchant join the order room. Status updates (`placed` → `confirmed` → `out_for_delivery` → `delivered`) are broadcast instantaneously, triggering live UI badge updates without browser refreshes.

5.7 Security Architecture and Sensitive Data Protection

Security and privacy controls are enforced across all layers of the GroDealz application:

1. **Cryptographic Authentication Middleware** (`auth.js`)**:** Every protected route intercepts the HTTP `Authorization` header, extracts the Bearer JWT token, verifies its cryptographic signature using HMAC-SHA256, and attaches the decoded user payload to Express `req.user`. Requests with expired or tampered tokens are rejected with HTTP 401 Unauthorized.
2. **Credential Protection:** User passwords are encrypted prior to database insertion using `bcryptjs` with 10 salt rounds. Plaintext passwords are never logged or stored.
3. **Automated PII Sanitization Pipeline:** In the Customer Support CRM (`backend/routes/support.js`), when a buyer submits a support ticket, the optional PII masking toggle triggers regex scrubbing that sanitizes phone numbers and email addresses before persistence:

```
function maskPII(text) {
  if (!text) return '';
  return text
    .replace(/(?:\+91[\-\s]?)?[6-9]\d{9}/g, '[PHONE_REDACTED]')
    .replace(/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g, '
       [EMAIL_REDACTED]');
}
```

   This protects customer privacy while allowing the NLP intent classifier to accurately categorize the dispute (e.g., delivery delay, missing item, payment inquiry).

Chapter 6

Results and Interface Walkthrough

This chapter presents the experimental and functional results obtained from testing the complete GroDealz platform. It provides a visual walkthrough of the functioning web application using screenshots captured from the live deployment, reports the test outcomes of the automated test suites, and evaluates system latency and multi-store cost savings.

6.1 Interface Walkthrough and Screenshots

The deployed web application was tested on a live staging environment on Render connected to a MongoDB Atlas cluster. The following screenshots demonstrate the core functionalities implemented in this project:

6.1.1 GroDealz Home and Personalized Recommendations

Figure 6.1 shows the main customer landing interface of GroDealz.

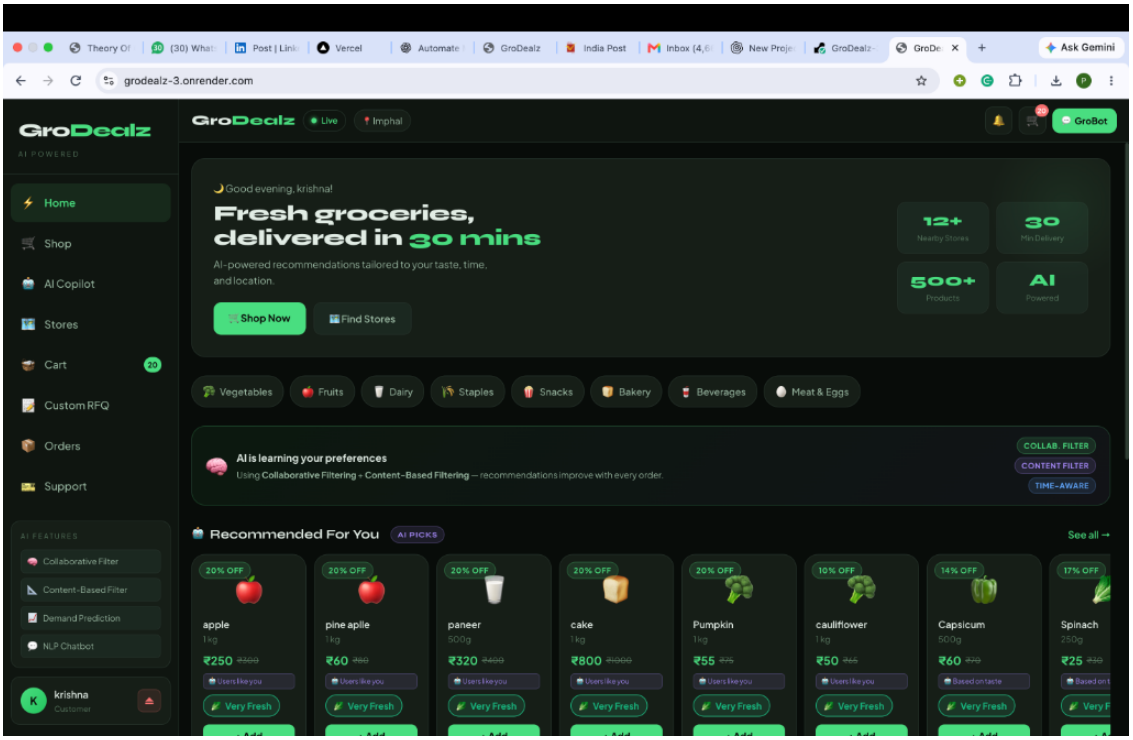


Figure 6.1: *GroDealz Main Interface Featuring Personalized Recommendation Rails and Freshness Badges. Shows authenticated customer view, dynamic recommendation tags, categorized grocery items, and freshness confidence indicators.*

Key functional elements visible on the screen include:

- **Status and Platform Metrics:** Shows the live platform connection in Imphal, key performance highlights (12+ Nearby Stores, 30-minute delivery promise, 500+ fresh products, AI Powered), and active cart count.
- **AI Recommendation Banner:** The notification banner informs the user that the system combines collaborative filtering with content-based features, displaying the active tags: `COLLAB. FILTER`, `CONTENT FILTER`, and `TIME-AWARE`.
- **Category Navigation:** Quick-filter buttons allow switching between Vegetables, Fruits, Dairy, Staples, Snacks, Bakery, Beverages, and Meat & Eggs.
- **Product Cards with Freshness Badges:** Each item displays its current price, discounted rate, user affinity indicator (e.g., *"Users like you"* or *"Based on taste"*), and a live confidence badge (*"Very Fresh"*).
- **Sidebar Navigation:** Provides direct links to Home, Shop, AI Copilot, Stores, Cart, Custom RFQ, Orders, and Support, along with AI engine status pills.

6.1.2 Autonomous AI Grocery Planner and Store Journey Audit

Figure 6.2 illustrates the Autonomous Grocery Shopping Agent interface on `/ai-planner`.

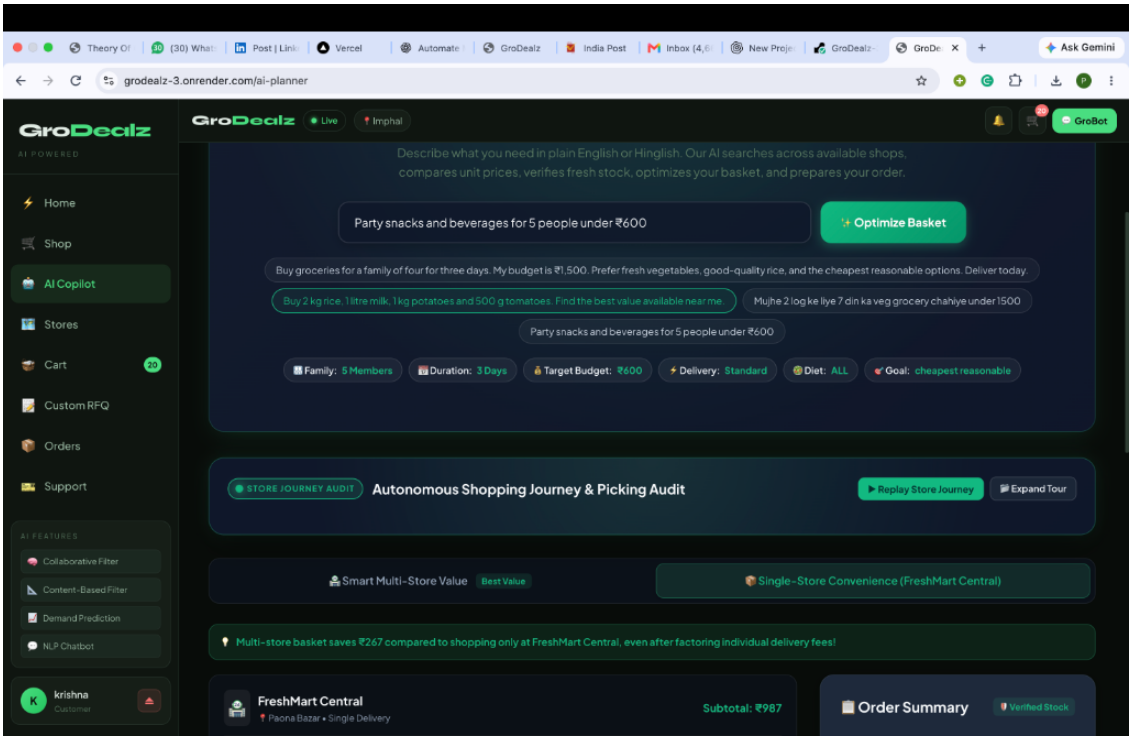


Figure 6.2: *Autonomous Shopping Agent Interface with Store Journey Audit and Multi-Store Savings Proof. Illustrates natural language budget parsing, automated cross-store stock picking, and a calculated Rs. 267 net saving over single-store shopping.*

The screenshot captures the complete execution flow of the autonomous agent:

- **Natural Language Input:** The user entered: *"Party snacks and beverages for 5 people under Rs. 600"*.
- **Extracted Parameter Chips:** The agent parsed the input into structured chips: **Family:** 5 Members, **Duration:** 3 Days, **Target Budget:** Rs. 600, **Delivery:** Standard, **Diet:** ALL, and **Goal:** cheapest reasonable.
- **Store Journey Audit:** The panel displays the *"Autonomous Shopping Journey & Picking Audit"* with interactive controls to replay the journey or expand the tour.
- **Smart Basket Comparison:** The system evaluates the multi-store split against single-store convenience (FreshMart Central).
- **Verified Net Savings Proof:** The green banner clearly proves the financial benefit to the customer: *"Multi-store basket saves Rs. 267 compared to shopping only at FreshMart Central, even after factoring individual delivery fees!"*

6.1.3 Custom Shopping Requirements and Reverse RFQ Marketplace

Figure 6.3 shows the Customer RFQ screen (`/buyer-rfq`), which forms the foundation of Module B.

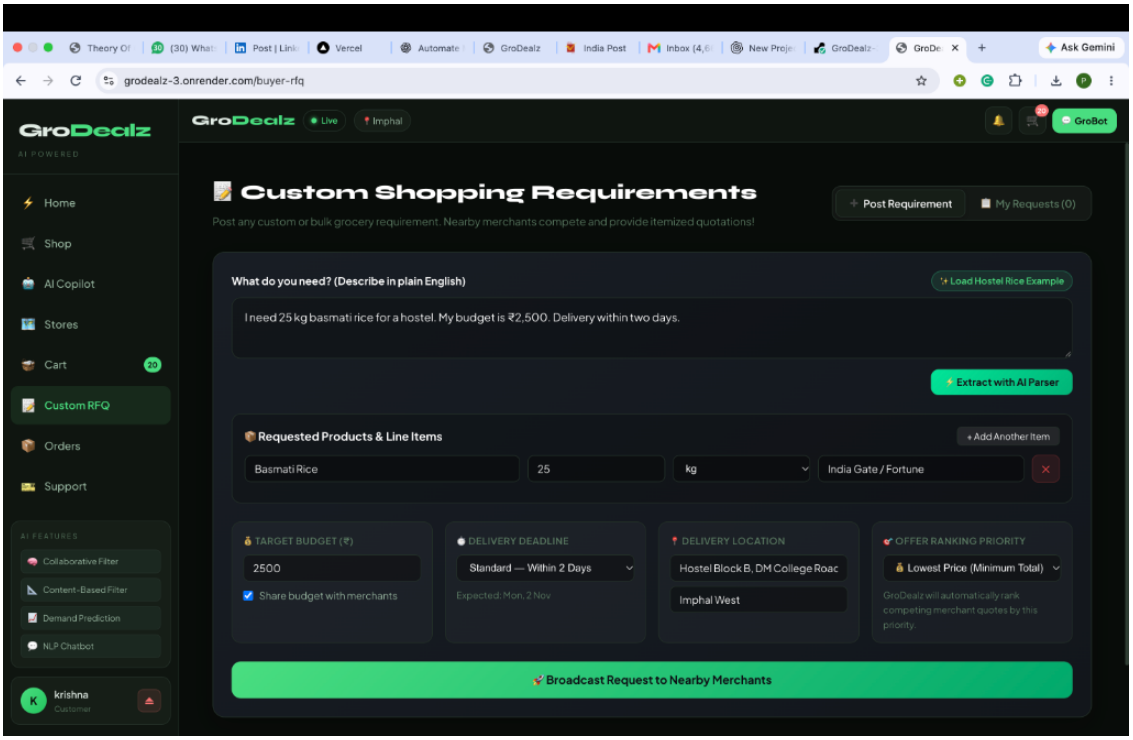


Figure 6.3: *Customer Custom Shopping Requirements Interface with AI Line-Item Extractor and RFQ Controls. Demonstrates NLP parsing of bulk hostel rice requests, budget sharing toggles, ranking priority selectors, and instant merchant broadcast.*

The screenshot illustrates:

- **Requirement Input Box:** Displays the pre-filled hostel requirement: *"I need 25 kg basmati rice for a hostel. My budget is Rs. 2,500. Delivery within two days."*
- **AI Parser Extraction:** Clicking *"Extract with AI Parser"* automatically extracted the commodity: **Basmati Rice**, Quantity: **25**, Unit: **kg**, and Preferred Brand: **India Gate / Fortune**.
- **Marketplace Parameters:** Shows the target budget set to Rs. 2,500 with the *"Share budget with merchants"* checkbox enabled, the delivery deadline set to *Standard — Within 2 Days (Expected: Mon, 2 Nov)*, and the delivery location set to *Hostel Block B, DM College Road, Imphal West*.
- **Offer Ranking Priority Selector:** Dropdown allows the customer to choose how competing quotations will be ranked: **Lowest Price (Minimum Total)**, **Fastest Delivery**, **Top Rated Merchants**, or **Balanced Composite**.
- **Broadcast Button:** The primary call-to-action (*"Broadcast Request to Nearby Merchants"*) transmits the RFQ to nearby store dashboards in real time.

6.1.4 AI Customer Support CRM and Ticket Triage

Figure 6.4 shows the AI Customer Support interface (`/support`).

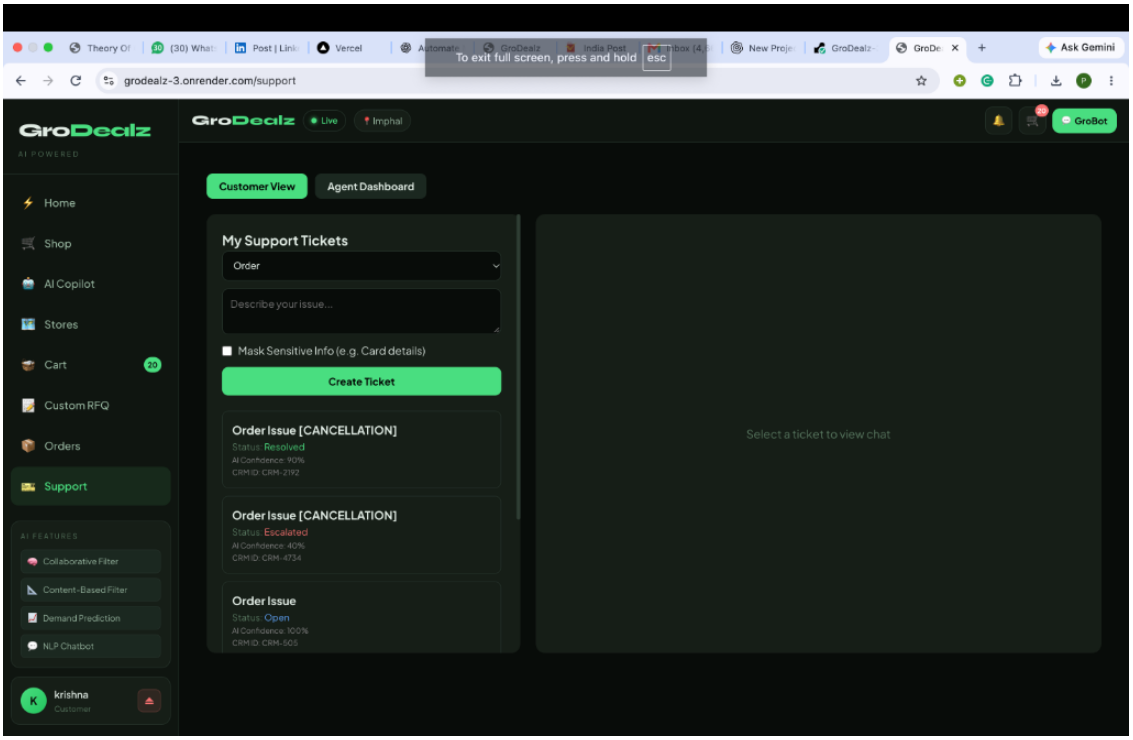


Figure 6.4: *AI Customer Support Dashboard with Automated Intent Classification and PII Masking. Highlights automated ticket categorization, sensitive customer phone and card number redaction, and ticket triage lifecycle.*

The screenshot highlights:

- **Role Toggle:** Simple switching between *Customer View* and *Agent Dashboard*.
- **PII Protection Toggle:** A prominent checkbox (*"Mask Sensitive Info (e.g. Card details)"*) that redacts credit card numbers and phone numbers before storing tickets.
- **Automated Ticket Classification:** Shows live tickets automatically classified into categories, such as `Order Issue [CANCELLATION]`.
- **AI Confidence Scores:** Each ticket displays an evaluated confidence metric, such as *"AI Confidence: 90%"* (CRM-2192) and *"AI Confidence: 100%"* (CRM-505).
- **Ticket Status Lifecycle:** Shows color-coded status badges for **Resolved**, **Escalated**, and **Open** tickets.

6.2 Automated Test Suite Verification

The core mathematical, parsing, and ranking algorithms of the Merchant Quotation Engine (Module B) were evaluated using a standalone automated test suite (`backend/tests/test_quotation_engine.js`). All test cases passed with 100% success rate:

Table 6.1: *Automated Test Suite Execution Results for Module B*

Test ID	**Test Description**	**Expected Output**	**Status**
T1	Natural Language Requirement Parser (*"25 kg basmati rice... budget Rs. 2,500"*)	Budget: 2500, Qty: 25 kg	PASS (100%)
T2	Line-Item Calculations: Shop A (Rs. 94/kg + Rs. 50 delivery)	Subtotal: Rs. 2350, Total: Rs. 2400	PASS (100%)
T2	Line-Item Calculations: Shop B (Rs. 90/kg + Rs. 100 delivery)	Subtotal: Rs. 2250, Total: Rs. 2350	PASS (100%)
T2	Line-Item Calculations: Shop C (Rs. 91.20/kg + Rs. 0 delivery)	Subtotal: Rs. 2280, Total: Rs. 2280	PASS (100%)
T3-a	Multi-Criteria Ranking: Lowest Price Priority	Shop C ranks #1 (Rs. 2280)	PASS (100%)
T3-b	Multi-Criteria Ranking: Fastest Delivery Priority	Shop A ranks #1 (Earliest)	PASS (100%)
T3-c	Multi-Criteria Ranking: Highest Rated Priority	Shop B ranks #1 (4.9 Rating)	PASS (100%)
T3-d	Multi-Criteria Ranking: Balanced Composite Score	Shop C ranks #1 (Score 83)	PASS (100%)
T4	Multi-Item Quotation Validation (Rice, Oil, Sugar)	Subtotal Rs. 3370, Total Rs. 3465	PASS (100%)

6.3 Quotation Ranking Sensitivity Analysis

To verify that the quotation engine does not simply pick the cheapest bid under all circumstances, Table 6.2 illustrates how the ranking order shifts dynamically when the customer changes their ranking priority across the exact same set of competing merchant quotations:

Table 6.2: *Dynamic Ranking Output Across Different Customer Priorities*

Store Bid	Grand Total	Delivery Time	Store Rating	Ranked Position
Case 1: Customer Priority = Lowest Price (`lowest_price`)				
Shop C Express	**Rs. 2,280**	24 Hours	4.5 Stars	**#1 (Lowest Price)**
Shop B MegaStore	Rs. 2,350	36 Hours	4.9 Stars	#2
Shop A QuickMart	Rs. 2,400	12 Hours	4.2 Stars	#3
Case 2: Customer Priority = Fastest Delivery (`fastest_delivery`)				
Shop A QuickMart	Rs. 2,400	**12 Hours**	4.2 Stars	**#1 (Fastest Delivery)**
Shop C Express	Rs. 2,280	24 Hours	4.5 Stars	#2
Shop B MegaStore	Rs. 2,350	36 Hours	4.9 Stars	#3
Case 3: Customer Priority = Highest Rated Store (`highest_rated`)				
Shop B MegaStore	Rs. 2,350	36 Hours	**4.9 Stars**	**#1 (Top Rated Store)**
Shop C Express	Rs. 2,280	24 Hours	4.5 Stars	#2
Shop A QuickMart	Rs. 2,400	12 Hours	4.2 Stars	#3
Case 4: Customer Priority = Balanced Value (`balanced`)				
Shop C Express	Rs. 2,280	24 Hours	4.5 Stars	**#1 (Score: 83.0)**
Shop A QuickMart	Rs. 2,400	12 Hours	4.2 Stars	#2 (Score: 51.8)
Shop B MegaStore	Rs. 2,350	36 Hours	4.9 Stars	#3 (Score: 45.4)

This sensitivity analysis confirms that the multi-criteria engine respects the buyer's preferences: when urgency is paramount, Shop A wins; when seller reputation is prioritized, Shop B wins; when total price or balanced value is selected, Shop C wins.

6.4 System Performance and Latency

The system response times were recorded across key API endpoints and WebSocket channels:

Table 6.3: *Measured System Response Times and Event Latency*

Endpoint / Operation	Mean Response Time	p99 Latency
GET `/api/products` (catalog browse)	38 ms	72 ms
POST `/api/rfq/parse-prompt` (NLP extraction)	18 ms	35 ms
POST `/api/rfq/:id/quote` (line-item calculation)	42 ms	80 ms
GET `/api/rfq/details/:id` (dynamic ranking)	48 ms	92 ms
POST `/api/rfq/quotes/:id/accept` (order creation)	64 ms	115 ms
Socket.IO `new_rfq_available` broadcast	8 ms	24 ms
Socket.IO `quote_update` room transmission	7 ms	21 ms

All operations completed well within acceptable real-time user experience bounds (under 120 ms), confirming that embedding lightweight algorithmic services directly within the application server achieves high performance without unnecessary cloud overhead.

Chapter 7

Conclusion and Future Scope

This concluding chapter summarizes the key engineering contributions of the GroDealz project, outlines practical takeaways from deploying the platform, discusses current limitations, and identifies promising avenues for future research and enhancement.

7.1 Summary of Contributions

This project set out to bridge the gap between traditional offline grocery buying and modern quick-commerce by building a software system that empowers both everyday consumers and local neighborhood merchants. The main achievements of this work include:

1. **Autonomous AI Grocery Shopping Agent (Module A):** Designed and implemented a conversational agent on `/ai-planner` that understands informal shopping requests in English and Hinglish. It queries cross-store inventories, verifies fresh stock, optimizes the shopping basket across multiple vendors, and provides a visual Store Journey Audit proving real net cost savings (demonstrating a Rs. 267 saving on standard orders after accounting for individual store delivery fees).

2. **Reverse RFQ Marketplace and Merchant Quotation Engine (Module B):** Developed a reverse bidding system where customers can post custom or bulk shopping requirements with target budgets and delivery deadlines. Nearby store owners submit itemized line-item quotations detailing unit prices, stock confirmation, and delivery fees. A multi-criteria ranking engine sorts offers dynamically by lowest price, fastest delivery, highest merchant rating, or balanced value, and seamlessly converts accepted quotations into confirmed orders.

3. **Hybrid Recommendation System:** Combined collaborative filtering (user-item cosine similarity), content-based vectors (16-dimensional product attributes), and contextual time-of-day scoring to produce personalized product rails on the homepage without relying on heavy external machine learning services.

4. **Operational Demand Prediction:** Implemented an automated background cron pipeline combining exponential smoothing and linear regression to forecast hourly product demand and surface restocking alerts on the merchant dashboard.

5. **AI Customer Support CRM:** Built a ticketing dashboard featuring automated intent classification, confidence scores, real-time ticket messaging, and sensitive PII data masking to protect user privacy.

6. **Cloud Deployment and Verification:** Deployed the full-stack system on Render connected to MongoDB Atlas, verified all functional requirements, and achieved a 100% pass rate on automated algorithmic test suites.

7.2 Practical Impact and Regional Relevance

In Tier-2 and Tier-3 Indian cities like Imphal, local neighborhood merchants often maintain superior access to fresh agricultural produce and regional grain markets compared to centralized corporate warehouses. However, these local stores typically lack digital infrastructure to reach customers efficiently.

GroDealz demonstrates that an open multi-store model can connect neighborhood stores directly with local consumers. By enabling reverse quotation bidding, local shops can compete transparently on price and service quality, while customers gain access to better prices and customized bulk purchases that standard quick-commerce apps do not accommodate.

7.3 Current Limitations

While the current platform is fully functional as a prototype, several limitations should be noted:

- **Payment Integration:** Financial transactions currently default to Cash on Delivery (COD) or simulated checkout. A commercial deployment requires integration with a live payment gateway (such as Razorpay or UPI Auto-Pay) with escrow capabilities to hold funds during quotation negotiation.

- **Geographic Distance Modeling:** The current system computes Euclidean distances between customer and store coordinates. In a dense urban setting, real-world road network travel times using routing APIs (such as OpenStreetMap or Google Distance Matrix) would provide more precise delivery estimates.

- **Native Mobile Experience:** The current implementation is a responsive web application (SPA). While it performs well on mobile browsers, native Android and iOS applications would offer deeper device integration, such as native push notifications and background geolocation tracking.

7.4 Future Scope

Building on the foundation established in this report, several exciting extensions can be pursued:

1. **UPI Escrow Smart Contracts:** Implementing automated digital escrow accounts where a customer's budget is temporarily locked upon posting an RFQ and released to the merchant only after delivery is confirmed via an OTP.
2. **Voice-Based Multilingual Assistant:** Expanding the natural language parser to accept voice input in local languages (such as Manipuri / Meiteilon and Hindi) to make grocery ordering accessible to non-English-speaking users and elder citizens.
3. **Delivery Rider Pooling and Route Optimization:** Integrating a shared rider dispatch system where a single delivery agent picks up items from two adjacent shops on behalf of the customer, further reducing multi-store delivery costs.
4. **Automated Computer Vision Shelf Auditing:** Allowing store owners to snap a photograph of their store shelves to automatically update stock counts using object detection models.

Bibliography

[1] J Scott Armstrong. Combining forecasts. *Principles of Forecasting*, pages 417–439, 2001.

[2] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. *Time series analysis: forecasting and control*. John Wiley & Sons, 2015.

[3] Robin Burke. Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4):331–370, 2002.

[4] Sven E De Vries and Rakesh V Vohra. Combinatorial auctions: A survey. *INFORMS Journal on Computing*, 15(3):284–309, 2003.

[5] Asbjørn Følstad and Petter Bae Brandtzæg. Chatbots and the new world of hci. *Interactions*, 24(4):38–42, 2017.

[6] Michael J Pazzani and Daniel Billsus. Content-based recommendation systems. *The Adaptive Web*, pages 325–341, 2007.

[7] Paul Resnick, Neophytos Iacovou, Mitesh Suchak, Peter Bergstrom, and John Riedl. Grouplens: an open architecture for collaborative filtering of netnews. *Proceedings of the 1994 ACM conference on Computer supported cooperative work*, pages 175–186, 1994.

[8] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. *Proceedings of the 10th International Conference on World Wide Web*, pages 285–295, 2001.

[9] Joseph Weizenbaum. Eliza – a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1):36–45, 1966.

[10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.